

卡路里智慧管家 重要程式與維護說明

Vue 3 × PHP × MySQL

專案：calorie-vue | 整理日期：2026-09-10

本文件以目前工作目錄的實際程式碼為依據，整理核心 API、Vue 頁面、資料庫設計與維護注意事項。適合用來理解專案、準備報告或交接。程式片段為原碼節錄，長行僅為排版折行，並非完整原始碼清單。

建議閱讀順序

先看整體架構，再沿著「首頁加入食物 → API 儲存 → MySQL → 會員中心」的流程閱讀。最後查看部署差異與維護清單。

02 架構與資料流程

03 API 共用層與登入狀態

04 API 端點速查

05 飲食儲存與資料隔離

06 食物搜尋與分頁

07 Vue 首頁與互動寫法

08 會員中心、BMI 與共享狀態

09 體重、趨勢及其他頁面

10 MySQL 資料表與初始化

11 Demo 與部署注意事項

12 執行、驗證與交接清單

查核範圍：src、backend、tests、package.json、vite.config.js、GitHub Actions。未讀取或收錄本機資料庫密碼；未連線檢查正式資料庫或線上網站。

02 | 架構與資料流程

前端做什麼

Vue 3 負責畫面、表單與即時計算；Vue Router 4 切換頁面；Vite 5 處理開發與打包。Bootstrap Icons 提供圖示。共用登入狀態使用模組層級 ref，並未使用 Pinia。

後端做什麼

backend/api/*.php 接收 JSON、確認登入及 CSRF、驗證欄位，再透過 PDO 存取 MySQL。前端不直接連線資料庫。

真實模式的完整流程

HomeView.vue → apiRequest() → /api/records.php → Session 取得會員 ID → 查詢 foods 並重算營養 → 交易寫入 meal_records / meal_items → 回傳 JSON → RecordsView.vue 讀取顯示。

Demo 模式的流程

Vue 頁面 → apiRequest() → demoApiRequest() → 瀏覽器 localStorage。畫面共用，但不呼叫 PHP，也不寫入 MySQL。

原始碼節錄：src/main.js，第 11-17 行

```
11 const bootstrap = async () => {  
12   const { initializeAuth } = useAuth()  
13   await initializeAuth()  
14   createApp(App).use(router).mount('#app')  
15 }  
16  
17 bootstrap()
```

啟動時先等待 initializeAuth()，再掛載 App 與路由，讓重新整理後的會員狀態先恢復。App.vue 只組合 NavBar 與 RouterView，實際內容分散在各 views。

核心檔案導航

src/api.js：請求封裝；src/auth.js：會員狀態；src/progress.js：統計函式；src/demo.js：展示資料；
backend/api/bootstrap.php：API 共用安全與回應函式。

03 | API 共用層與登入狀態

原始碼節錄：src/api.js · 第 25-37 行

```
25 const method = (options.method || 'GET').toUpperCase()
26 const headers = { Accept: 'application/json', ...options.headers }
27 const requestOptions = { ...options, method, headers, credentials: 'include' }
28
29 if (options.body !== undefined) {
30   headers['Content-Type'] = 'application/json'
31   requestOptions.body = JSON.stringify(options.body)
32 }
33 if (!['GET', 'HEAD'].includes(method) && csrfToken) {
34   headers['X-CSRF-Token'] = csrfToken
35 }
36
37 let response
```

credentials: include 讓瀏覽器帶上 Session Cookie。body 自動轉 JSON；非 GET / HEAD 操作在取得 Token 後加入 X-CSRF-Token。API 網址使用 VITE_API_BASE_URL，未設定則為 /api。

原始碼節錄：backend/api/bootstrap.php · 第 68-86 行

```
68 function requireCsrf(): void
69 {
70   $submittedToken = $_SERVER['HTTP_X_CSRF_TOKEN'] ?? '';
71   if ($submittedToken === '' || !hash_equals(csrfToken(), $submittedToken)) {
72     respond(['ok' => false, 'message' => '安全驗證已失效，請重新整理後再試'], 419);
73   }
74 }
75
76 function requireUserId(): int
77 {
78   $userId = (int) ($_SESSION['user_id'] ?? 0);
79   if ($userId <= 0) {
80     respond(['ok' => false, 'message' => '請先登入'], 401);
81   }
82
83   return $userId;
84 }
85
86 function findUserById(PDO $db, int $userId): ?array
```

requireUserId() 從伺服器 Session 讀取會員 ID；requireCsrf() 使用 hash_equals 比對 Token。登入與註冊成功時會更新 Session ID 並產生新 Token；me.php 可恢復登入狀態。

錯誤怎麼處理

api.js 將連線錯誤、無效 JSON 與 API 失敗轉成 ApiError，保留 HTTP status，Vue 再顯示 error.message。
bootstrap.php 對未處理例外回傳一般 500 訊息，詳細例外留在伺服器 log。

04 | API 端點速查

路徑皆以 /api/ 為前綴。以下為實際 PHP 端點；修改型請求使用 JSON body。

POST register.php

公開註冊。username、password、email、fullName、phone、acceptedTerms。成功回傳 user、csrfToken。
· HTTP 201；帳號或信箱重複為 409。

POST login.php

公開登入。username、password、remember。password_verify 驗證雜湊；成功建立 Session 並回傳 user、csrfToken。

GET me.php / POST logout.php

me 回傳 authenticated，已登入另帶 user 與 csrfToken。logout 需要登入與 CSRF，清除 Session 和 Cookie。

GET / PATCH profile.php

需登入。GET 取得 user；PATCH 更新 fullName、email、phone 或完整 BMI 輸入組，並驗證 CSRF。

GET foods.php

公開食物搜尋。q、category、after、meta。回傳 foods、hasMore、nextCursor；meta=1 加上 categories、recommendations。

GET / PUT / DELETE records.php

需登入。GET 回傳 records；PUT 傳 mealDate、meals；DELETE 傳 id 或 all:true。PUT / DELETE 需 CSRF。

GET / PUT / PATCH / DELETE weights.php

需登入。GET 回傳 records、target；PUT 傳 date、weight；PATCH 傳 target；DELETE 傳 id。所有修改均需 CSRF。

常見 HTTP 狀態

400：JSON 錯誤；401：未登入或登入失敗；404：紀錄不存在；405：方法錯誤；409：重複資料；419：CSRF 失效；422：欄位不合法；500：伺服器例外。

登入及註冊端點目前沒有呼叫 requireCsrf()；不可將「所有 POST 都有 CSRF」當成既有實作。正式部署時也應另行評估登入限流，目前登入程式未實作。

05 | 飲食儲存：最重要的後端程式

原始碼節錄：backend/api/records.php · 第 93-103 行

```
93 $db->beginTransaction();
94 try {
95     $query = $db->prepare('INSERT INTO meal_records (user_id, meal_date) VALUES (?, ?)
96         ON DUPLICATE KEY UPDATE id = LAST_INSERT_ID(id), updated_at = CURRENT_TIMESTAMP');
97     $query->execute([$userId, $date]);
98     $id = (int) $db->lastInsertId();
99     $db->prepare('DELETE FROM meal_items WHERE record_id = ?')->execute([$id]);
100     $insert = $db->prepare('INSERT INTO meal_items (record_id, meal_type, food_name, weight_g, calories, protein_g, fat_g, carbs_
101         g)
102         VALUES (?, ?, ?, ?, ?, ?, ?, ?)');
103     foreach ($items as $item) $insert->execute([$id, ...$item]);
104     $db->commit();
105 }
```

同一會員、同一天由唯一鍵限制成一筆。ON DUPLICATE KEY UPDATE 取得既有 ID，再刪除舊明細並寫入整天新內容；交易確保整批成功，例外則 rollback。這不是追加單一食物的 API。

熱量不能相信前端

前端可傳入 calories，但後端實際只根據 name、weight_g 查詢 foods，再以 weight / 100 重新計算。熱量取整數，蛋白質、脂肪與碳水取一位小數，重量先取兩位小數。

資料隔離

GET 以 r.user_id 篩選；單筆 DELETE 同時比對 id 與 user_id；all:true 也只刪目前會員。前端不得指定其他會員 ID。

輸入限制

日期介於 1900-01-01 與台北今天；餐別只能早餐、午餐、晚餐；每餐最多 100 項；每項重量 0.01-10000 克；至少一項食物。食物名稱須存在於 foods。

操作上要注意

同一天再次儲存會取代整天內容；不同分頁同時編輯時，後儲存者可能覆蓋先前結果。首頁清空只改畫面草稿，空清單不能 PUT 儲存；刪除已存資料需從紀錄功能呼叫 DELETE。

06 | 食物搜尋與分頁

原始碼節錄：backend/api/foods.php · 第 26-30 行

```
26 $query = $db->prepare('SELECT ' . $fields . ' FROM foods WHERE ' . implode(' AND ', $where) . ' ORDER BY id LIMIT 21');
27 $query->execute($params);
28 $foods = $query->fetchAll();
29 $hasMore = count($foods) > 20;
30 $foods = array_slice($foods, 0, 20);
```

後端多取一筆 (21 筆) 判斷是否還有下一頁，再回傳前 20 筆。游標使用最後一筆 id；下一頁查 id > after，避免一次下載全部清單。

原始碼節錄：src/views/HomeView.vue · 第 143-151 行

```
143 watch([searchQuery, activeCategory], () => {
144   clearTimeout(foodSearchTimer)
145   ++foodSearchVersion
146   foodDatabase.value = []
147   hasMoreFoods.value = false
148   foodsError.value = ''
149   foodsLoading.value = true
150   foodSearchTimer = setTimeout(() => loadFoods(), 250)
151 }, { flush: 'sync' })
```

watch 偵測關鍵字與分類變動，250 毫秒內連續輸入只保留最後一次排程。foodSearchVersion 遞增；loadFoods 收到回應時比對版本，避免較慢的舊搜尋蓋掉新結果。

捲動載入與推薦

距離清單底部 48 px 內呼叫 loadFoods(true)。已有請求、無下一頁或沒有游標時不重送。meta=1 取得分類與推薦食物，之後不反覆要求。

效能與維護限制

foods.php 對 name / aliases 使用 LIKE 包含搜尋，會跳脫 % 與 _ 等萬用字元。category,id 索引由 setup.php 建立；前後皆有 % 的子字串搜尋仍可能掃描大量資料。現有改善主要降低傳輸及渲染量，不能直接宣稱通過大規模效能測試。

07 | Vue 首頁：狀態、計算與儲存

看懂 .vue 的三個區塊

script setup 放狀態與函式；template 描述畫面與事件；style 控制外觀。ref 是會更新畫面的資料，computed 是依其他資料推導的結果，watch 用來在狀態變動後載入資料或排程搜尋。

原始碼節錄：src/views/HomeView.vue · 第 153-167 行

```
153 const calculatedPreview = computed(() => {
154   if (!currentSelectedFood.value || inputWeight.value <= 0) {
155     return { calories: 0, protein: 0, fat: 0, carbs: 0 }
156   }
157   const ratio = inputWeight.value / 100
158   const food = currentSelectedFood.value
159   return {
160     calories: Math.round(food.calories * ratio),
161     protein: Number((food.protein_g * ratio).toFixed(1)),
162     fat: Number((food.fat_g * ratio).toFixed(1)),
163     carbs: Number((food.carbs_g * ratio).toFixed(1))
164   }
165 })
166
167 const selectSearchResult = food => {
```

選取食物與重量改變時，computed 自動算出預覽。食物資料以每 100 克為基準，例如白米飯每 100 克為 130 kcal，150 克顯示 195 kcal。這是專案食物資料的換算範例。

原始碼節錄：src/views/HomeView.vue · 第 296-301 行

```
296 await apiRequest('records.php', { method: 'PUT', body: { mealDate: mealDate.value, meals: meals.value } })
297 showSaveSuccess.value = true
298 } catch (error) {
299   mealError.value = error.message
300 } finally {
301   saving.value = false
```

畫面資料的生命週期

meals 保存早餐、午餐、晚餐陣列；addFood / removeFoodFromMeal 修改草稿；computed 累加總熱量與營養素，進度圈最多 100%，超過目標變紅色。saving 防止重複儲存。

日期與帳號切換

watch([mealDate, currentUser], loadMeals) 會清空目前草稿並重新讀取該日期紀錄。切換日期前須先儲存；loadVersion 避免舊回應覆蓋新日期。離開頁面會移除點擊監聽器與搜尋計時器。

08 | 會員中心、BMI 與共享狀態

RecordsView.vue

載入飲食紀錄、切換日期 / 近 7 天 / 近 30 天、刪除單筆或全部紀錄，並提供個人資料與 BMI 表單。

DietTrends 與 WeightTracker 嵌入此頁。

原始碼節錄：backend/api/profile.php · 第 87-90 行

```
87 $bmi = round($weight / (($height / 100) ** 2), 1);
88 $sexAdjustment = $sex === 'male' ? 5 : -161;
89 $bmr = (int) round(10 * $weight + 6.25 * $height - 5 * $age + $sexAdjustment);
90 $dailyCalories = (int) round($bmr * $activity);
```

以上是專案實作公式的說明：BMI 使用公斤與公尺平方；bmr 使用體重、身高、年齡與性別常數，再乘 activity 得到 dailyCalories。這裡描述程式行為，不是個人醫療或飲食建議。

送出欄位要成組

PATCH 只要包含 height、weight、sex、birthDate、activity 任一欄，後端就會驗證整組。前端應一次送齊五項，不能只更新 weight。限制：100-250 cm、20-300 kg、18-100 歲；活動係數只接受 1.4、1.6、1.8、2.0、2.2。

原始碼節錄：src/auth.js · 第 13-22 行

```
13 const applyUser = (user, isDemo = false) => {
14   profile.value = user
15   isLoggedIn.value = true
16   currentUser.value = user.username
17   displayName.value = user.fullName || user.username
18   demoMode.value = isDemo
19   dailyCalorieTarget.value = Number(user.dailyCalories) || 2000
20 }
21
22 const clearUser = () => {
```

API 回傳 user 後，applyUser 更新共用 ref。首頁與 NavBar 共用 dailyCalorieTarget；無有效個人目標時預設 2000。getProfile 回傳淺拷貝，編輯後仍須透過 updateProfile 存入後端。

權限與估算界線

路由只有 /records 標示 requiresAuth。路由守衛是使用者介面控制，真正資料權限仍由 PHP 判斷。專案 README 說明估算供一般成人參考，功能文案與公式修改時應一起維護。

09 | 體重、趨勢與其他 Vue 頁面

WeightTracker.vue + weights.php

GET 載入體重與目標；PUT 新增或更新同一天體重；PATCH 設定目標；DELETE 刪除指定紀錄。修改成功後重新 GET，讓畫面跟後端一致。體重與目標允許 20-300 kg。

容易忽略：兩種體重資料互不連動

weight_records 是趨勢紀錄；users.weight 是 BMI 個人資料。weights.php 沒有更新 users，也不會重新計算每日熱量。因此儲存新體重後，BMI 與熱量目標不會自動更新；需另在 BMI 表單儲存。

原始碼節錄：src/progress.js，第 14-18 行

```
14 export function recordedAverage(series) {  
15   const recorded = series.filter(point => point.value !== null)  
16   return { count: recorded.length, average: recorded.length ? Math.round(recorded.reduce((sum, point) => sum + point.value, 0) / r  
    ecoreded.length): null }  
17 }  
18 export function weightSummary(records, today = new Date()) {
```

平均只計算有紀錄的天數，未紀錄為 null，並非吃了 0 kcal。dailySeries 補齊日期；weightSummary 比較最早與最新體重，本週差異則使用週一前最後一筆作基準，資料不足回傳 null。

TrendChart.vue / DietTrends.vue

TrendChart 使用 SVG 座標、折線段與資料點畫圖；遇到 null 會中斷線段。DietTrends 組合日期序列及週平均，提供 7 / 30 天趨勢。圖表不依賴第三方圖表套件。

其他頁面清單

LoginView / RegisterView：輸入驗證、API 登入 / 註冊及導頁。SportView：響應式輪播；SportDetailView：依路由 category 讀取 locations.js。TeacherView：靜態營養師卡片。BookingView：表單驗證與成功狀態。NavBar：導覽、會員名稱、熱量目標與登出。

預約還沒完成後端功能

BookingView.submit 只設定 submitted=true，沒有 API、資料表或通知流程。TeacherView 的價格、評分、評論數是寫在程式的靜態內容，不代表即時查證或真實交易系統。

10 | MySQL 資料表與初始化

users

會員帳號、雜湊密碼、聯絡資料、BMI 輸入與每日熱量。username、email 各有唯一限制。

foods

公開食物主檔，名稱唯一；營養素以 100 克為基準；aliases 提供別名搜尋，source / source_id 保存來源。

meal_records → meal_items

主檔記 user_id、meal_date，明細保存餐別、名稱、重量與當次計算出的營養。user_id + meal_date 唯一。

weight_records / weight_goals

體重歷史以 user_id + record_date 唯一；目標以 user_id 為主鍵，每人一個目標。

原始碼節錄：backend/database/schema.sql，第 45-54 行

```
45 CREATE TABLE IF NOT EXISTS meal_records (  
46   id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,  
47   user_id BIGINT UNSIGNED NOT NULL,  
48   meal_date DATE NOT NULL,  
49   updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
50   UNIQUE KEY user_meal_date (user_id, meal_date),  
51   FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
52 ) ENGINE=InnoDB;  
53  
54 CREATE TABLE IF NOT EXISTS weight_records (  
55   user_id BIGINT UNSIGNED NOT NULL,  
56   record_date DATE NOT NULL,  
57   weight BIGINT UNSIGNED NOT NULL,  
58   goal BIGINT UNSIGNED NOT NULL,  
59   UNIQUE KEY user_record_date (user_id, record_date),  
60   FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
61 ) ENGINE=InnoDB;
```

外鍵與歷史快照

刪除會員會連帶刪除其飲食與體重資料；刪除飲食主檔會刪明細。meal_items 儲存當時的營養數值，不會因 foods 修改而自動變更；再次儲存該日則會使用新主檔重新計算。

初始化實際行為

setup.php 限 CLI 執行，要求 local.php 存在；執行 schema.sql，再補 aliases 與 category_id 索引並匯入種子。schema 固定使用 calorie_db，不能只改連線設定就假設初始化會建立另一個資料庫。

維護食物資料

種子合併為 241 筆；重跑以名稱補缺，不覆蓋已存在食物，但 setup 另含發酵乳名稱 / 別名更新。來源說明見 backend/database/FOOD-SOURCES.md。飲料也以克計，不能直接假設毫升等於克。資料庫改名前應注意首頁尚未儲存的舊名稱可能無法通過驗證。

11 | Demo 與正式部署的差異

原始碼節錄：src/demo.js，第 206-211 行

```
206 export const canHandleDemoRequest = (path, staticPreview = false) => {
207   const endpoint = new URL(path, 'https://demo.local/').pathname.split('/').pop()
208   return isDemoSessionActive() || (staticPreview && endpoint === 'foods.php')
209 }
210
211 export async function demoApiRequest(path, options = {}) {
```

只要 localStorage 的 Demo Session 仍 active，就會把請求導向 Demo 資料層，即使本機 PHP 已啟動也一樣。需要驗證真實 API 時，先登出 Demo，再用一般會員登入。

資料存在哪裡

Demo 使用 calorie-demo-session-v1 與 calorie-demo-state-v1，包含範例會員、7 天飲食、5 筆體重及食物目錄。重新載入範例會重設 Demo 資料；清除瀏覽器資料也會失去紀錄，無跨裝置同步。

Demo 不能替代後端測試

Demo 的 profile 更新直接合併欄位，並未完整重做 PHP 驗證與 BMI 計算；其他端點也有較寬鬆驗證。畫面在 Demo 正常，不代表真實 API 權限、資料庫或計算已通過測試。

部署設定

vite.config.js 的 base 固定 /calorie-vue/；改網站子路徑時要一起更新。開發時 /api 代理到 127.0.0.1:8000。Actions 在 main 推送後 npm ci、npm run build，再上傳 dist；流程沒有執行 tests。

深層路由重新整理

router 使用 createWebHistory。現有工作流程只發布 dist，未看到 404 fallback 設定。正式主機要將前端路徑重寫至 index.html；GitHub Pages 的 /records 等深層網址直接開啟可能 404，需實測並補 fallback 或評估 hash 路由。

跨站 API 與 Cookie

只設定 VITE_API_BASE_URL 不足以完成跨站登入。現有 PHP 沒有 CORS 回應設定，Cookie 為 SameSite=Lax；跨來源要配置允許來源與 credentials，真正跨站還需評估 Cookie 策略及 HTTPS。同源反向代理通常較容易維持現有架構。

設定檔與上線維護

local.php 已列入 .gitignore；資料庫密碼不可放進前端 VITE_*。正式環境需使用適當資料庫權限與 HTTPS。PHP 內建伺服器是本機開發用途。

12 | 執行、驗證與交接清單

本機啟動步驟

1. 安裝 Node.js 與 XAMPP，啟動 MySQL。
2. 參考 backend/config/local.php.example 建立 local.php 並填入本機設定。
3. npm install ; npm run db:setup。
4. 一個終端機執行 npm run api ; 另一個執行 npm run dev。
5. 開啟 <http://localhost:5173/calorie-vue/>，註冊一般會員。

package.json 將 PHP 寫死為 C:\xampp\php\php.exe；若安裝位置不同須修改腳本。npm run build 產出 dist；npm run preview 是靜態前端預覽，不能當作 PHP API 已啟動的證明。

這次實際完成的檢查

node tests/progress.mjs：通過，涵蓋營養剩餘 / 超標、缺資料日期、平均及體重週界線。
node tests/home-search.mjs：通過，涵蓋延遲搜尋、過時回應、選取食物、分類與卸載清理。
node tests/demo.mjs：通過，檢查食物目錄、範例引用與 Demo 儲存程式；不是完整瀏覽器端到端測試。

尚未執行的整合驗證

本次為文件整理，未執行需真實 MySQL 的 tests/records.mjs，也未驗證線上部署或執行完整 UI 測試。該整合測試需先建立資料庫，會建立兩個測試會員，驗證登入、CSRF、計算、同日更新、隔離與刪除，最後清理測試資料。

建議手動驗收

登入一般會員 → 加入白米飯 150 克 → 儲存 → 會員中心核對 195 kcal → 同帳號另一瀏覽器確認同步 → 另一帳號確認不可看到前者資料。另檢查切換日期未存草稿、同日覆寫、體重目標、登出與重新整理。

交接時先講清楚的五件事

- ① 真實 MySQL 與 Demo 完全分開。
- ② 同日飲食為整天覆寫。
- ③ 體重趨勢與 BMI 分開儲存。
- ④ 預約目前只有前端。
- ⑤ 部署必須處理 API、Cookie 與深層路由。

來源索引：每頁程式片段列出檔案及行號；補充依據為 README.md、package.json、vite.config.js、.github/workflows/static.yml、backend/database/schema.sql、setup.php 與 tests/*.mjs。行號以整理當下工作目錄為準，後續修改可能移動。